



Codenix — Assistant IA conversationnel

Rapport technique complet du projet

Version 1.0 — Septembre 2026 — Rédigé en français
Chat IA (Groq) + Q&A sur PDF (RAG) + historique persistant + CI/CD

Sommaire

1. Présentation générale
2. Technologies utilisées
3. Architecture globale
4. Architecture détaillée du backend
5. Architecture détaillée du frontend
6. Documentation API (Swagger / OpenAPI)
7. Diagramme de cas d'utilisation
8. Diagramme de classes (domaine)
9. Diagrammes de séquence
10. Base de données (MongoDB + FAISS)
11. DevOps (CI/CD, qualité, charge)
12. Déploiement
13. Stratégie de versions et montée de version
14. Technologies à adopter
15. Roadmap des fonctionnalités + Conclusion

1. Présentation générale

Codenix est une application web full-stack : frontend **Next.js**, backend **FastAPI**. Un utilisateur authentifié (Clerk) dialogue avec un modèle d'IA (Groq), peut joindre un **PDF à son message** pour des réponses fondées sur le document (RAG), et retrouve son **historique** persisté.

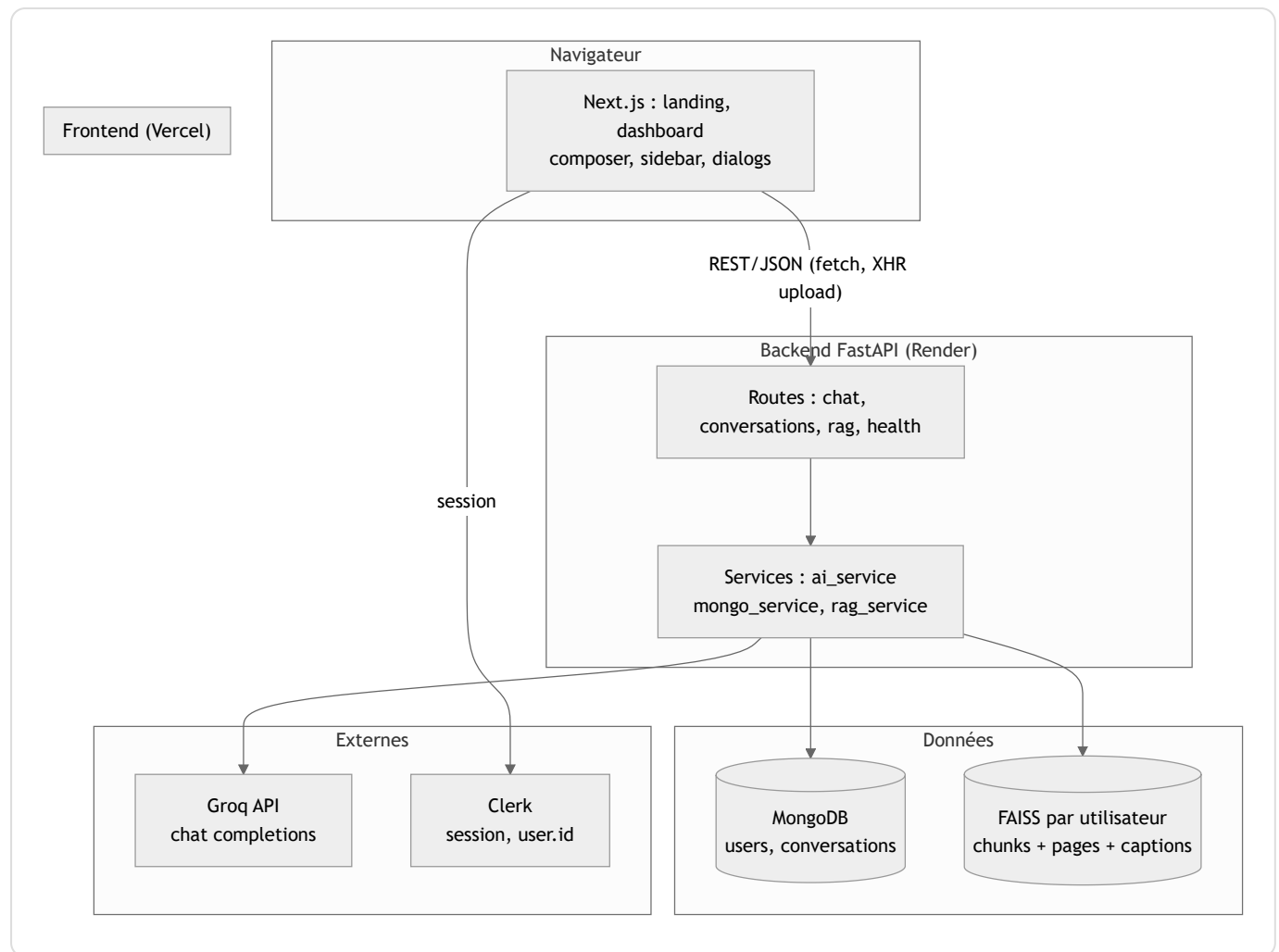
Fonctionnalités principales

- Chat temps réel (typewriter), Markdown riche, coloration syntaxique + copie.
- Envoi combiné *PDF* + *message* avec progression 0–100 %.
- Modes **Ask / Agent**, sélecteur de modèle, palette Ctrl+K.
- Thèmes sombre/clair, sidebar redimensionnable, suppression avec confirmation.

2. Technologies utilisées

Couche	Technologie	Version	Rôle
Frontend	Next.js (App Router)	15.5.26	SSR/SSG, routage, groupes de routes
Frontend	React + TypeScript	19 / 5	UI, typage strict
Frontend	Tailwind CSS	4 (tokens oklch)	Thème sombre/clair
Frontend	Clerk	6.24	Auth (groupe dashboard uniquement)
Frontend	PrismLight	15.6.1	Code (14 langages)
Backend	FastAPI + Uvicorn	0.141.1 / 0.54.0	API REST asynchrone + Swagger
Backend	Pydantic	2.13.5	Validation (400/422)
IA	Groq (AsyncGroq)	1.7.0	Completions + fallback multi-modèles
Base	MongoDB (PyMongo)	4.18.2 / 7	Users + conversations (Atlas ou Docker)
RAG	FAISS + MiniLM + LangChain	1.15.1 / 6.1.0 / 1.4.2	Index vectoriel par utilisateur
PDF	pypdf	6.19.0	Texte, pages, figures, captions
Tests	pytest / Vitest / Playwright / Locust	9.1.1 / 4 / 1.63	30 + 26 + 6 tests, charge 10k
DevOps	GitHub Actions, SonarQube, Docker, Dependabot	—	CI, qualité, versions
Déploiement	Vercel (front) + Render (back)	—	Serverless + serveur persistant

3. Architecture globale



4. Architecture détaillée du backend

Client (navigateur)

Middleware
CORS + GZip + lifespan
(index)

api/routes/
chat · conversations · rag ·
health

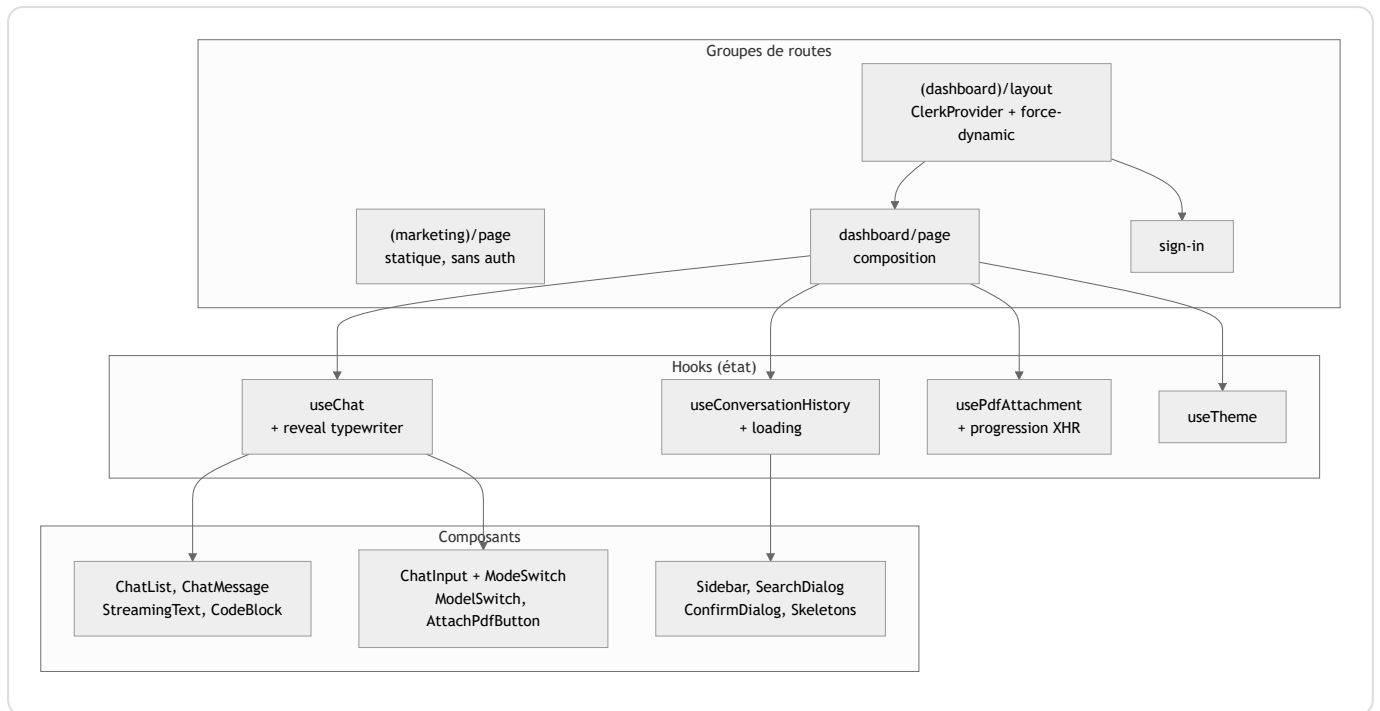
schemas/ (Pydantic)
ChatRequest,
ConversationIn...
→ 400/422 automatiques

services/
ai_service : AsyncGroq +
fallback + sanitize
mongo_service : pool 50,
ping 30s, index
rag_service : FAISS + cache
mtime

Groq · MongoDB · FAISS
disque

`main.py` ne contient que la fabrique (CORS, GZip, lifespan). Les routes sont fines, les schémas valident, les services portent le métier. Erreurs mappées : 503 (dépendances), 502 (provider avec cause), 500 (filet).

5. Architecture détaillée du frontend



Client API typé (`lib/api.ts` , `ApiError` avec le `detail` backend), helpers purs testés (`lib/messages.ts`), classes sémantiques de thème (aucune couleur en dur).

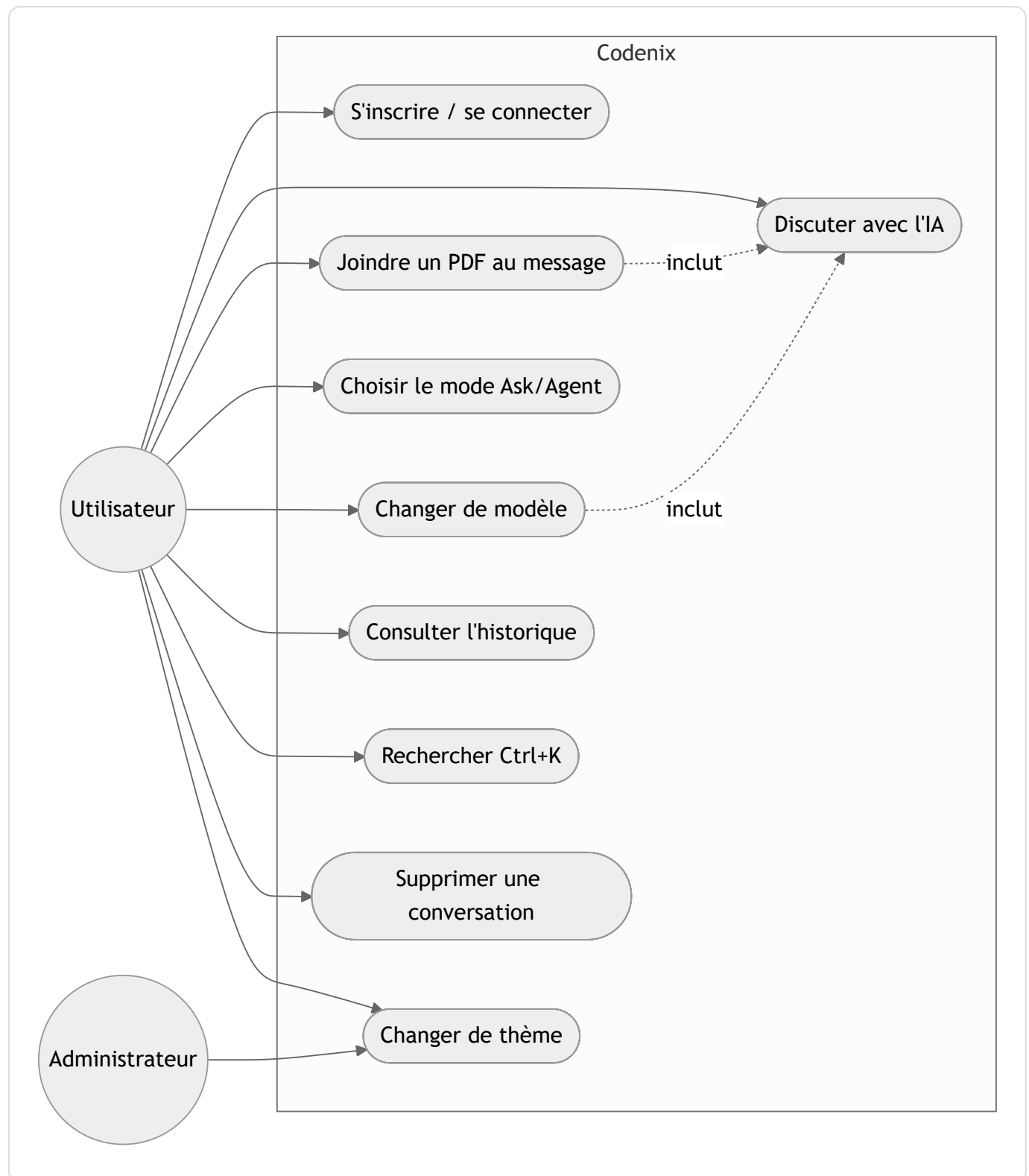
6. Documentation API (Swagger / OpenAPI)

FastAPI expose nativement une documentation interactive, enrichie (titre *Codenix API v1.0.0*, tags par domaine) :

- `GET /docs` — Swagger UI : essayer chaque endpoint depuis le navigateur.
- `GET /redoc` — variante ReDoc, lecture seule.
- `GET /openapi.json` — schéma machine (11 paths) pour générer des clients.

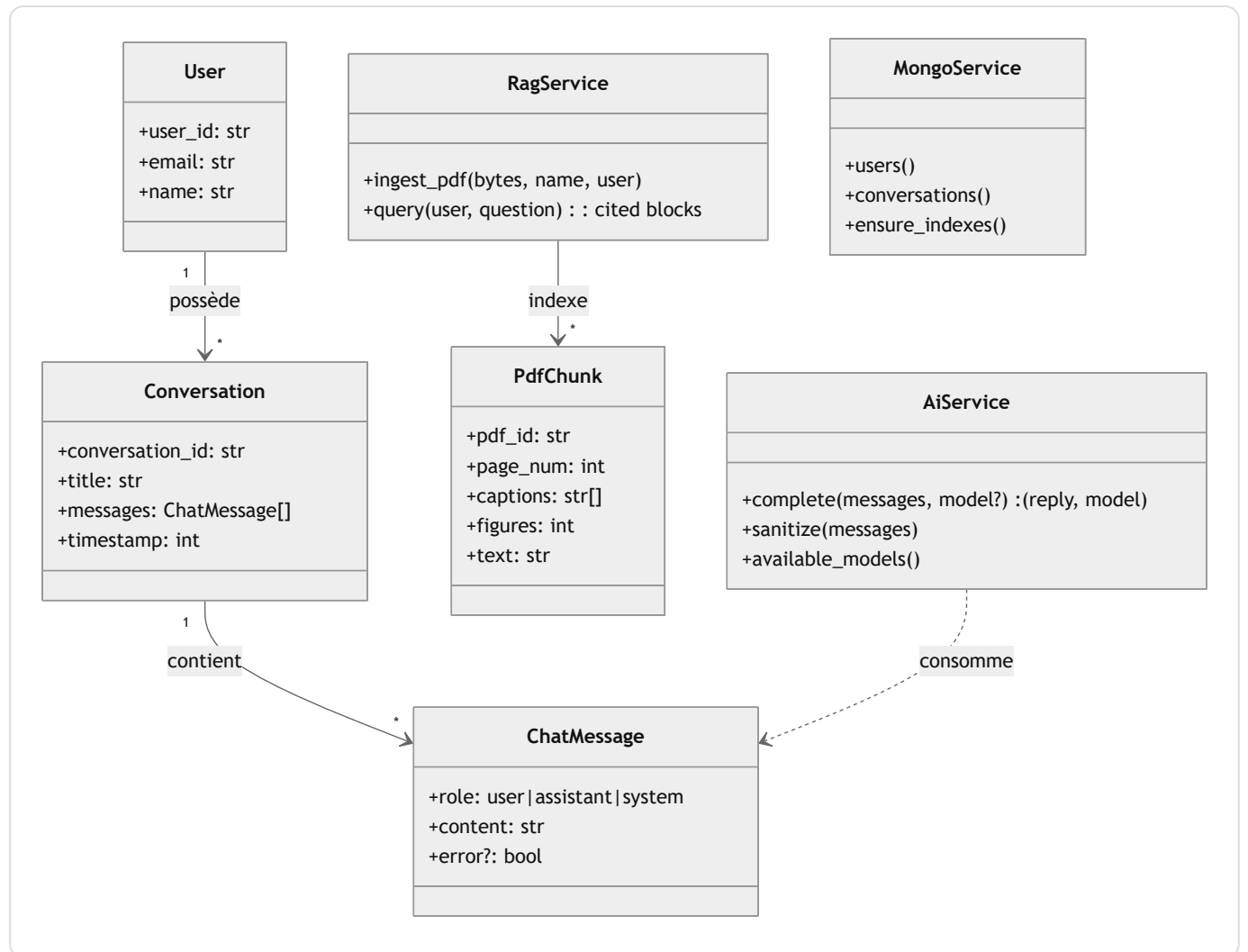
Tag	Endpoints couverts
chat	<code>/ask</code> , <code>/api/models</code>
conversations	<code>store_user</code> , <code>save/get/delete_conversation</code>
rag	<code>upload_pdf</code> , <code>query_rag</code> , <code>list_chunks</code> , <code>vector_health</code>
health	<code>/health</code>

7. Diagramme de cas d'utilisation



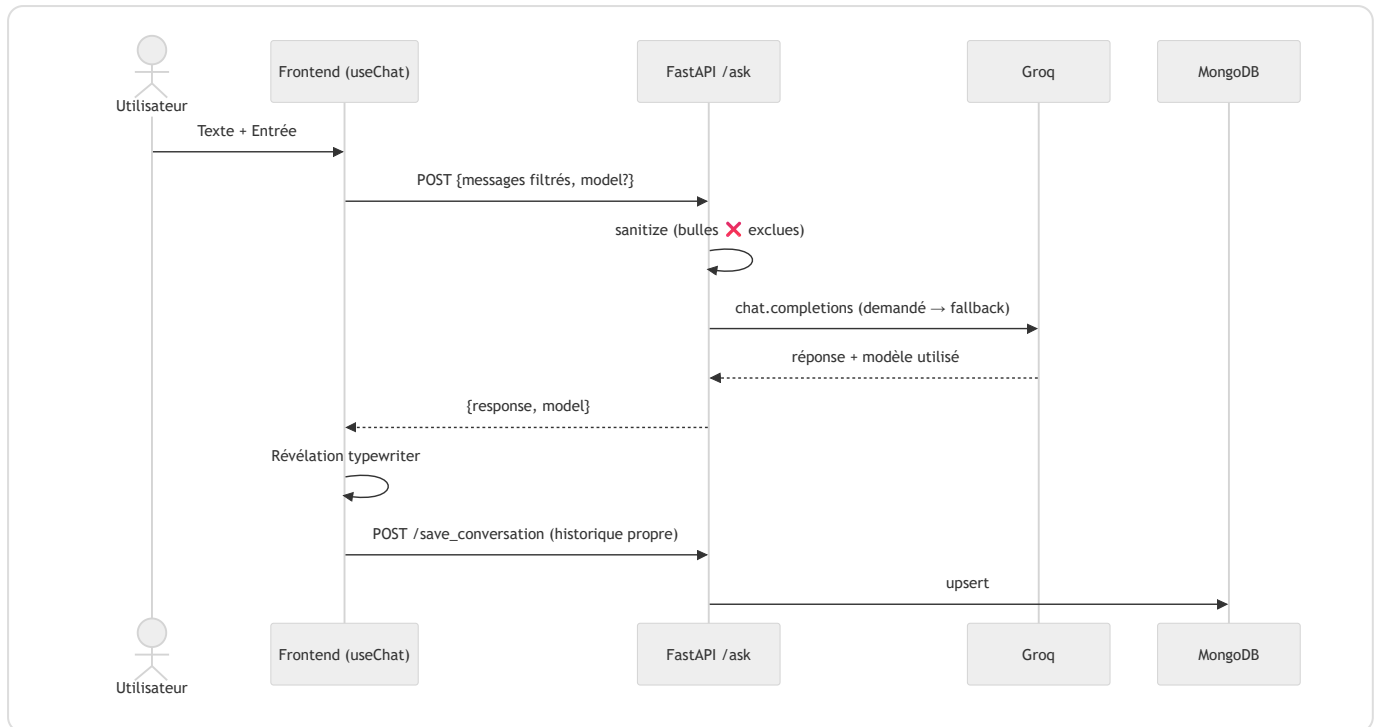
Acteur	Cas	Description
Utilisateur	Discuter	Message → réponse animée, formatée, persistée
Utilisateur	PDF + message	Chip d'attente, upload 0–100 %, réponse fondée sur le document
Utilisateur	Modes / modèle	Style Ask ou Agent, modèle commutable avec fallback
Utilisateur	Historique	Liste, recherche, suppression confirmée
Administrateur	Superviser	/health, logs, Sonar, Locust, versions

8. Diagramme de classes (domaine)

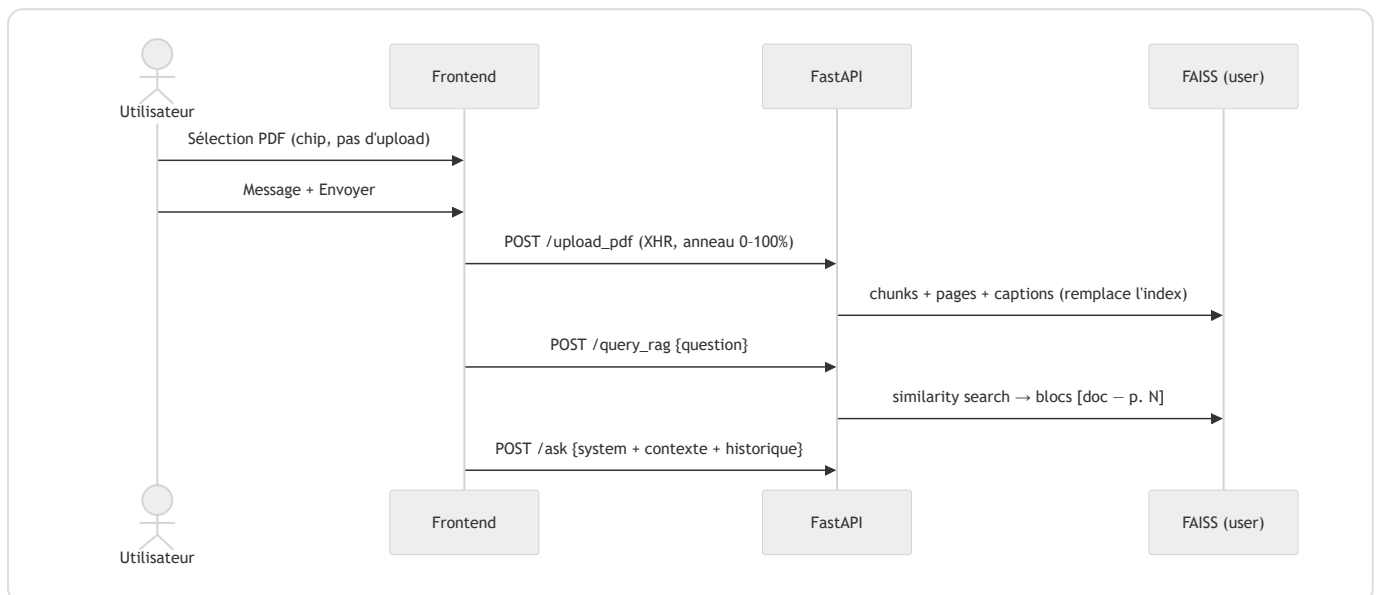


9. Diagrammes de séquence

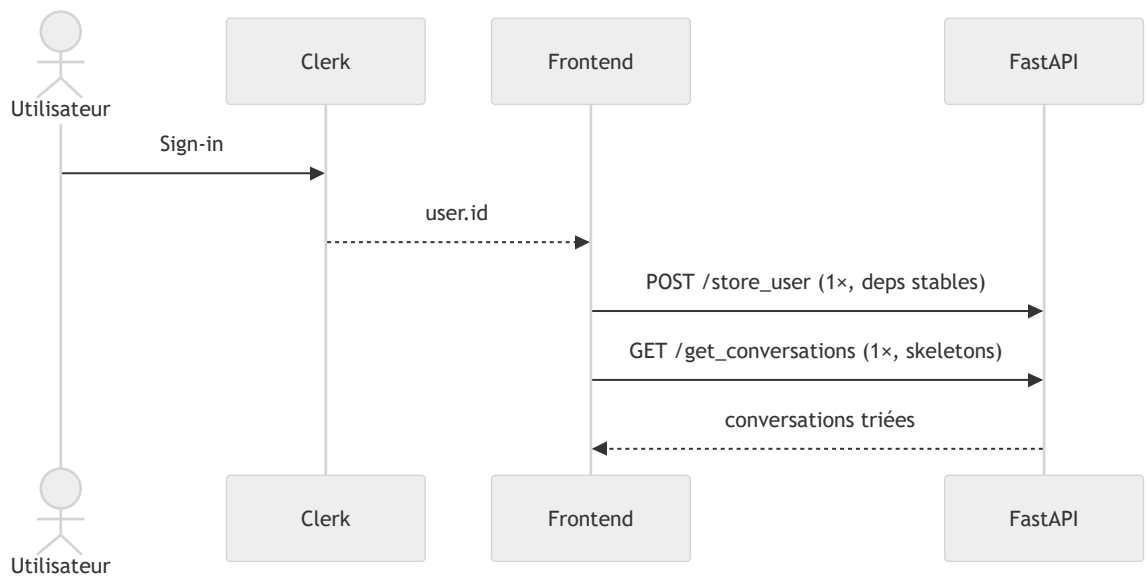
9.1 Envoi d'un message simple



9.2 Message avec PDF (RAG)



9.3 Connexion et historique

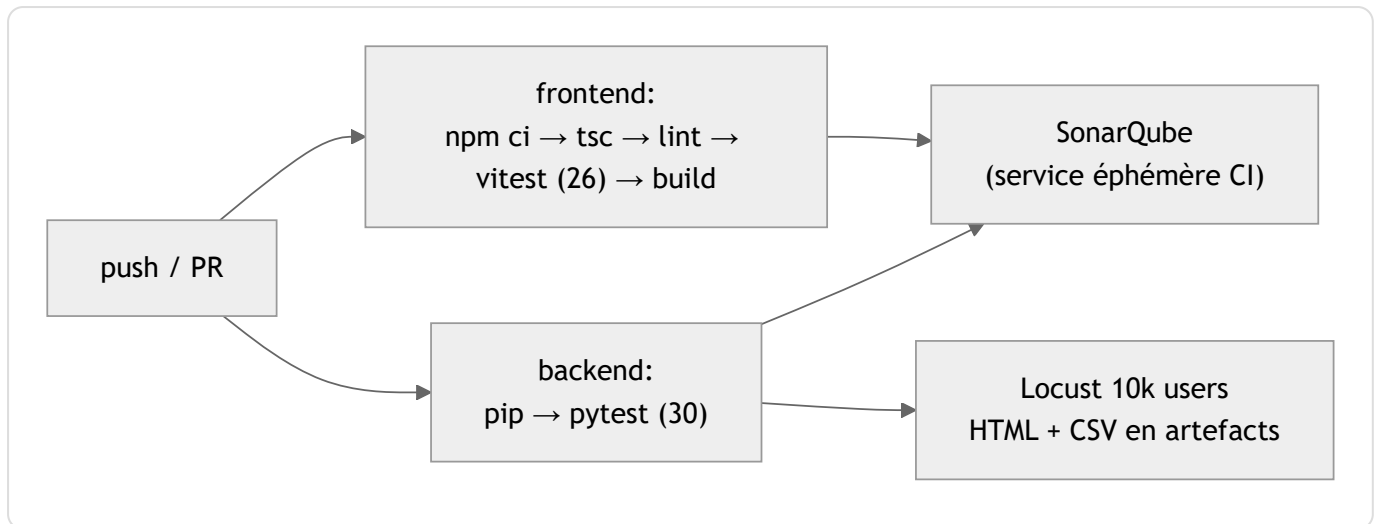


10. Base de données (MongoDB + FAISS)

Collection	Champs	Index
users	user_id, email, name	user_id unique
conversations	user_id, conversation_id, title, messages[], timestamp	(user_id, timestamp ↓)

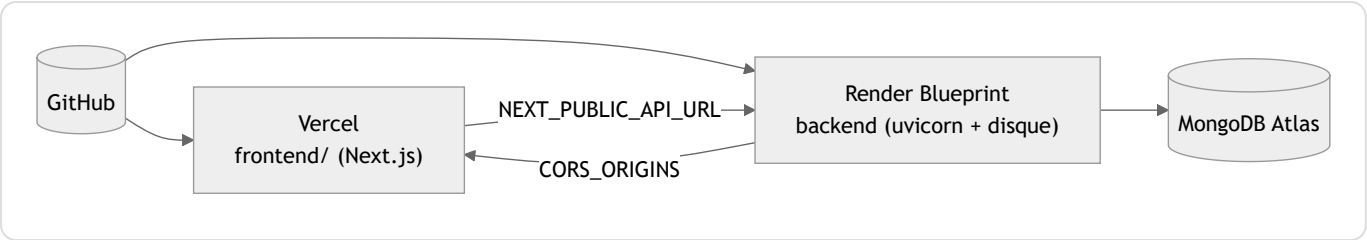
FAISS sur disque, isolé par utilisateur, cache mémoire invalidé par mtime. MongoDB : Atlas ou conteneur Docker (`docker compose up`).

11. DevOps (CI/CD, qualité, charge)



- **CI GitHub Actions + Dependabot** hebdomadaire (npm + pip, CI verte exigée).
- **Tests** : 30 pytest (mockés), 26 Vitest, 6 Playwright, Locust tags `ai / infra`.
- **Contraintes apprises** : plafond Windows `select()` 512 FD (workers + limit-concurrency), quota Groq (runs `--exclude-tags ai`).

12. Déploiement



Cible	Procédure	Variables
Backend (Render)	New → Blueprint (<code>render.yaml</code> : <code>build</code> , <code>{PORT:-8000}</code> , <code>disque /var/data</code> , <code>/health</code>)	GROQ_API_KEY, MONGO_URI, CORS_ORIGINS, FAISS_INDEX_PATH=/var/data
Frontend (Vercel)	Import, Root Directory <code>frontend</code> , Output vide (jamais <code>public</code>), Next ≥ 15.5.26	Clés Clerk ×2, NEXT_PUBLIC_API_URL
Clerk	Domaine Vercel + redirections (/dashboard)	—
Local/Docker	<code>docker compose up --build</code> (Mongo inclus)	backend/.env

13. Stratégie de versions et montée de version

SemVer strict (MAJEURE.MINEURE.PATCH) : APP_VERSION (frontend) et version FastAPI avancent ensemble ; tout changement visible = entrée Changelog (format Keep a Changelog : Added/Changed/Fixed).

Processus de montée de version des dépendances

- 1. **Audit** : npm outdated / pip list --outdated + npm audit ; Dependabot ouvre déjà les PRs chaque lundi (npm + pip, max 5, CI exigée).
- 2. **Branche dédiée** chore/upgrade-xxx : une famille à la fois (jamais framework + libs ensemble).
- 3. **Verrous** : package-lock.json et versions requirements.txt committés ; pas de latest flottant (leçon : Next 15.3.5 → 15.5.26 épinglé).
- 4. **Validation** : tests (pytest + Vitest + Playwright), next build local, revue Swagger (/docs), déploiement staging si risque.
- 5. **Montées planifiées** :

Dépendance	Actuel → Cible	Motif
Next.js	15.5.26 → 16 (évaluer)	breaking : tester Turbopack + cache
langchain-community	0.4.2 → packages standalone	Paquet en fin de vie (warning au boot)
Node.js	20 → 22 LTS	Aligner CI + runtime
React	19 (à jour)	Surveillance mineurs
torch / sentence-transformers	Épinglés, maj trimestrielle	Éviter les régressions d'embeddings

14. Technologies à adopter (recommandées)

Technologie	Usage	Priorité
Redis	Cache des chunks RAG + rate-limit par utilisateur (protège le quota Groq)	P0
SSE (Server-Sent Events)	Streaming temps réel des réponses au lieu du typewriter simulé	P0
Sentry (front + back)	Remontée d'erreurs production avec contexte	P0
S3 / MinIO	Stockage des PDF (multi-instances, au lieu du disque local)	P1
Atlas Search / Meilisearch	Recherche plein texte + hybride (BM25 + vecteurs) dans l'historique	P1
next-intl	Internationalisation FR/AR officielle	P1
PWA (manifest + service worker)	Installation mobile, icône, mode offline partiel	P2
Prometheus + Grafana	Métriques (latence /ask, 502/429, FD) et alertes	P2
Husky + pre-commit	Lint/tests avant chaque commit (même garde-fous qu'en CI)	P2

15. Roadmap des fonctionnalités + Conclusion

Fonctionnalité	Priorité	Notes
Upload multi-PDF simultanés	P0	L'index actuel est remplacé à chaque upload
Vision LLM des figures UML	P0	Nécessite une clé avec modèle vision (Llama 4 Scout)
Partage de conversation (lien) + export PDF/Markdown	P1	Token de partage, rendu existant réutilisable
Dossiers / tags + recherche plein texte	P1	Avec Atlas Search ci-dessus
Votes 👍👎 + analytics d'usage	P1	Boucle qualité des réponses
Entrée vocale (Whisper)	P2	Groq héberge whisper-large-v3
Tableau admin (quotas, coûts)	P2	Avec Prometheus ci-dessus

Conclusion. Codenix v1.0 livre un socle complet et testé : conversation robuste, RAG cité par pages, UI moderne, CI/CD avec qualité et charge, documentation Swagger intégrée et stratégie de versions. Prochaine étape conseillée : **SSE + Redis (P0)**, puis vision des figures.